Лабораторная работа №5: Основные модели

Мария Улитина

Table of contents

1. Введение **1**

2. Модель SIR **2**

3. Симуляция задачи ``Обедающие философы'' с помощью сетей Петри **2**
3.1. Подготовка окружения . . . 2
3.2. Параметры симуляции . . . 2
3.3. Классическая сеть (без арбитра) . . . 3
3.4. Сеть с арбитром . . . 7

4. Параметрический анализ SIR **9**

5. Анимация работы сети Петри для задачи ``Обедающие философы'' **9**
5.1. Подготовка окружения . . . 9
5.2. Параметры симуляции . . . 10
5.3. Создание анимации . . . 11
5.4. Сохранение анимации . . . 11

6. Модель Лотки-Вольтерры **12**

7. Сравнительный анализ классической сети и сети с арбитром **12**
7.1. Загрузка данных . . . 12
7.2. Построение графиков . . . 14
7.3. Сохранение результата . . . 17

1. Введение

Исследуются модели SIR и Лотки-Вольтерры.

2. Модель SIR

3. Симуляция задачи ``Обедающие философы'' с помощью сетей Петри

В этом скрипте мы моделируем классическую задачу синхронизации ``Обедающие философы'' с использованием сетей Петри. Сравниваются две модели: классическая (которая приводит к deadlock) и модифицированная с арбитром (которая предотвращает deadlock).

3.1. Подготовка окружения

Подключаем DrWatson для управления проектом и активируем его.

```
using DrWatson
@quickactivate "project"
```

Подключаем наш модуль с реализацией сетей Петри для философов.

```
include(srcdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
```

Подключаем библиотеки для работы с таблицами, CSV, графиками и случайностью.

```
using DataFrames, CSV, Plots, Random
gr(fmt=:png)
```

```
Plots.GRBackend()
```

3.2. Параметры симуляции

Количество философов --- 5.

```
N = 5
```

```
5
```

Максимальное время симуляции --- 50 единиц.

```
tmax = 50.0
```

```
50.0
```

3.3. Классическая сеть (без арбитра)

```
println("=== Классическая сеть (без арбитра) ===")
```

```
=== Классическая сеть (без арбитра) ===
```

Строим классическую сеть Петри для N философов.

```
net_classic, u0_classic, _ = build_classical_network(N)
```

```
(Main.DiningPhilosophers.PetriNet(20, 15, [-1 0 … 0 0; 0 -1 … 0 0; … ; 0 0 … 1 0;
```

Фиксируем seed для воспроизводимости результатов.

```
Random.seed!(456)
```

```
TaskLocalRNG()
```

Запускаем стохастическую симуляцию (алгоритм Гиллеспи).

```
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
```

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3	
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	...
2	0.174962	1.0	1.0	1.0	0.0	1.0	0.0	0.0	0.0	...
3	0.450131	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	...
4	0.997633	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
5	2.91918	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	...
6	2.97678	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
7	3.39601	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
8	4.49113	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
9	4.94763	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	...
10	5.16753	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0	...
11	5.25562	1.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	...
12	6.243	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	...
13	8.75498	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	...
14	9.52698	0.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	...
15	9.66133	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	...
16	10.9366	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0	...
17	11.308	0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0	...
18	11.5912	0.0	1.0	1.0	1.0	0.0	1.0	0.0	0.0	...
19	11.6521	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0	...
20	11.7669	0.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	...
21	12.0395	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
22	14.9816	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
23	15.6627	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
24	16.1644	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	...

Сохраняем результаты в CSV-файл.

```
CSV.write(datadir("dining_classic.csv"), df_classic)
```

```
"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab05/project/data/dining_classic.csv"
```

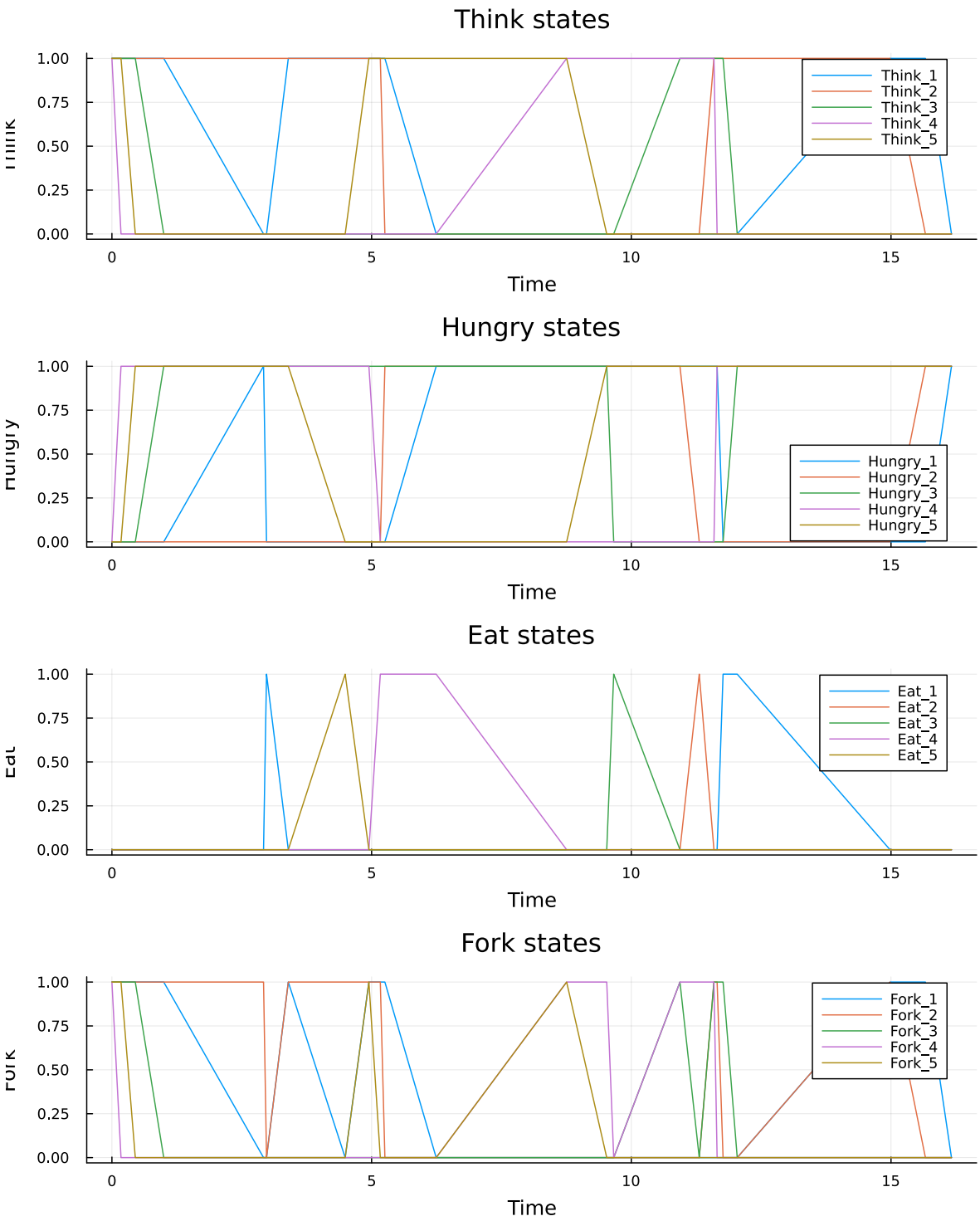
Проверяем, наступил ли deadlock.

```
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
```

```
Deadlock обнаружен: true
```

Строим графики эволюции маркировки.

```
plot_classic = plot_marking_evolution(df_classic, N)
```



Сохраняем графики в файл.

```
savefig(plotsdir("classic_simulation.png"))
```

```
"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab05/project/plots/classic_simulation.png"
```

3.4. Сеть с арбитром

```
println("\n=== Сеть с арбитром ===")
```

```
=== Сеть с арбитром ===
```

Строим модифицированную сеть с арбитром.

```
net_arb, u0_arb, _ = build_arbiter_network(N)
```

```
(Main.DiningPhilosophers.PetriNet(21, 15, [-1 0 … 0 0; 0 -1 … 0 0; … ; 0 0 … 1 1;
1 -1 … 1 1], [:Think_1, :Think_2, :Think_3, :Think_4, :Think_5, :Hungry_1, :Hungr
```

Запускаем стохастическую симуляцию.

```
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
```

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3	
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	...
2	0.124923	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	...
3	0.285503	0.0	1.0	1.0	1.0	0.0	1.0	0.0	0.0	...
4	0.367472	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0	...
5	0.946857	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	...
6	1.45161	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
7	2.59507	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
8	2.69306	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	...
9	3.29849	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
10	3.56091	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
11	4.29757	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
12	4.66568	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
13	7.08442	1.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	...
14	7.25198	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
15	8.76002	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
16	9.55978	1.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	...
17	9.67541	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	...
18	9.70523	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	...
19	9.88677	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	...
20	9.91366	0.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	...
21	10.2036	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	...
22	10.5081	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0	...
23	10.9861	0.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	...
24	11.9951	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	...
25	12.1165	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0	...
26	12.3187	0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0	...
27	12.3908	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	...
28	12.5664	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0	...
29	12.9798	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	...
30	18.7455	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
...	...	...	...	...	...	...	...	...	...	

Сохраняем результаты.

```
CSV.write(datadir("dining_arbiter.csv"), df_arb)
```

```
"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab05/project/data/dining_arbiter.csv"
```

Проверяем deadlock --- в сети с арбитром его быть не должно.

```
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
```

```
Deadlock обнаружен: false
```

Строим и сохраняем графики.

```
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))
```

```
"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab05/project/plots/arbiter_simulation.png"
```

4. Параметрический анализ SIR

5. Анимация работы сети Петри для задачи ``Обедающие философы''

Этот скрипт создаёт анимацию, которая показывает, как меняется маркировка сети Петри во времени. Анимация особенно наглядно демонстрирует возникновение deadlock.

5.1. Подготовка окружения

Подключаем DrWatson и активируем проект.

```
using DrWatson
@quickactivate "project"
```

Подключаем наш модуль с философами.

```
include(srcdir("DiningPhilosophers.jl"))
using .DiningPhilosophers

using .DiningPhilosophers:
    build_classical_network,
    build_arbiter_network,
    simulate_stochastic,
    detect_deadlock,
    plot_marking_evolution
```

```
WARNING: ignoring conflicting import of DiningPhilosophers.build_classical_networ
WARNING: ignoring conflicting import of DiningPhilosophers.build_arbiter_network
WARNING: ignoring conflicting import of DiningPhilosophers.simulate_stochastic in
WARNING: ignoring conflicting import of DiningPhilosophers.detect_deadlock into M
WARNING: ignoring conflicting import of DiningPhilosophers.plot_marking_evolution
```

Подключаем библиотеки для графиков и случайных чисел.

```
using Plots, Random
```

5.2. Параметры симуляции

Берём трёх философов, чтобы анимация была проще и нагляднее.

```
N = 3
```

```
3
```

Симулируем 30 единиц времени.

```
tmax = 30.0
```

```
30.0
```

Строим классическую сеть Петри.

```
net, u0, names = build_classical_network(N)
```

```
(Main.DiningPhilosophers.PetriNet(12, 9, [-1 0 … 0 0; 0 -1 … 1 0; … ; 0 -
1 … 1 0; 0 0 … 1 1], [:Think_1, :Think_2, :Think_3, :Hungry_1, :Hungry_2, :Hungry
```

Фиксируем seed, чтобы анимация получалась одинаковой при каждом запуске.

```
Random.seed!(123)
```

```
TaskLocalRNG()
```

Запускаем стохастическую симуляцию.

```
df = simulate_stochastic(net, u0, tmax)
```

	time	Think_1	Think_2	Think_3	Hungry_1	Hungry_2	Hungry_3	Eat_1	Eat_2	
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	
1	0.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	...
2	0.217198	1.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	...
3	0.255714	0.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0	...
4	0.577262	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0	...

5.3. Создание анимации

Макрос @animate пробегает по каждой строке таблицы результатов и для каждой строки рисует один кадр.

```
anim = @animate for row in eachrow(df)


    u = [row[col] for col in propertynames(row) if col != :time]


    bar(
        1:length(u),           # номера позиций по горизонтали
        u,                     # количество фишек по вертикали
        legend = false,        # легенда не нужна, и так много подписей
        ylims = (0, maximum(u0) + 1),  # фиксируем вертикальную шкалу
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )

    xticks!(1:length(u), string.(names), rotation = 45)
end
```

```
Animation("/tmp/jl_0jfSSE", ["000001.png", "000002.png", "000003.png", "000004.pn
```

5.4. Сохранение анимации

Собираем все кадры в GIF-файл с частотой 2 кадра в секунду.

```
gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
```

```
[ Info: Saved animation to /home/mmulitina/work/study/2026-1/backup/2026-
1-study-simulation-modeling/labs/lab05/project/plots/philosophers_simulation.gif
```

```
Plots.AnimatedGif("/home/mmulitina/work/study/2026-1/backup/2026-1-
study-simulation-modeling/labs/lab05/project/plots/philosophers_simulation.gif")
```

Выводим сообщение об успешном сохранении.

```
println("Анимация сохранена в plots/philosophers_simulation.gif")
```

```
Анимация сохранена в plots/philosophers_simulation.gif
```

6. Модель Лотки-Вольтерры

7. Сравнительный анализ классической сети и сети с арбитром

Этот скрипт загружает результаты двух симуляций (классической сети и сети с арбитром) и строит сравнительный график состояния ``Ест'' для всех пяти философов. График наглядно показывает разницу: в классической сети наступает deadlock, а в сети с арбитром --- нет.

7.1. Загрузка данных

Подключаем DrWatson и активируем проект.

```
using DrWatson
@quickactivate "project"
```

Подключаем библиотеки для работы с таблицами, CSV и графиками.

```
using DataFrames, CSV, Plots
```

Загружаем ранее сохранённые результаты симуляций.

```
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
```

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3	
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	...
2	0.124923	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	...
3	0.285503	0.0	1.0	1.0	1.0	0.0	1.0	0.0	0.0	...
4	0.367472	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0	...
5	0.946857	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	...
6	1.45161	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
7	2.59507	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
8	2.69306	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	...
9	3.29849	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
10	3.56091	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
11	4.29757	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
12	4.66568	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
13	7.08442	1.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	...
14	7.25198	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
15	8.76002	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	...
16	9.55978	1.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	...
17	9.67541	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	...
18	9.70523	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	...
19	9.88677	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	...
20	9.91366	0.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	...
21	10.2036	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	...
22	10.5081	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0	...
23	10.9861	0.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	...
24	11.9951	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	...
25	12.1165	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0	...
26	12.3187	0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0	...
27	12.3908	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	...
28	12.5664	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0	...
29	12.9798	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	...
30	18.7455	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...
...	...	...	...	...	...	...	...	...	...	

Количество философов --- 5.

```
N = 5
```

```
5
```

Формируем список столбцов, соответствующих состоянию ``Ест'' для каждого философа. В этих столбцах значения: 1 --- философ ест, 0 --- не ест.

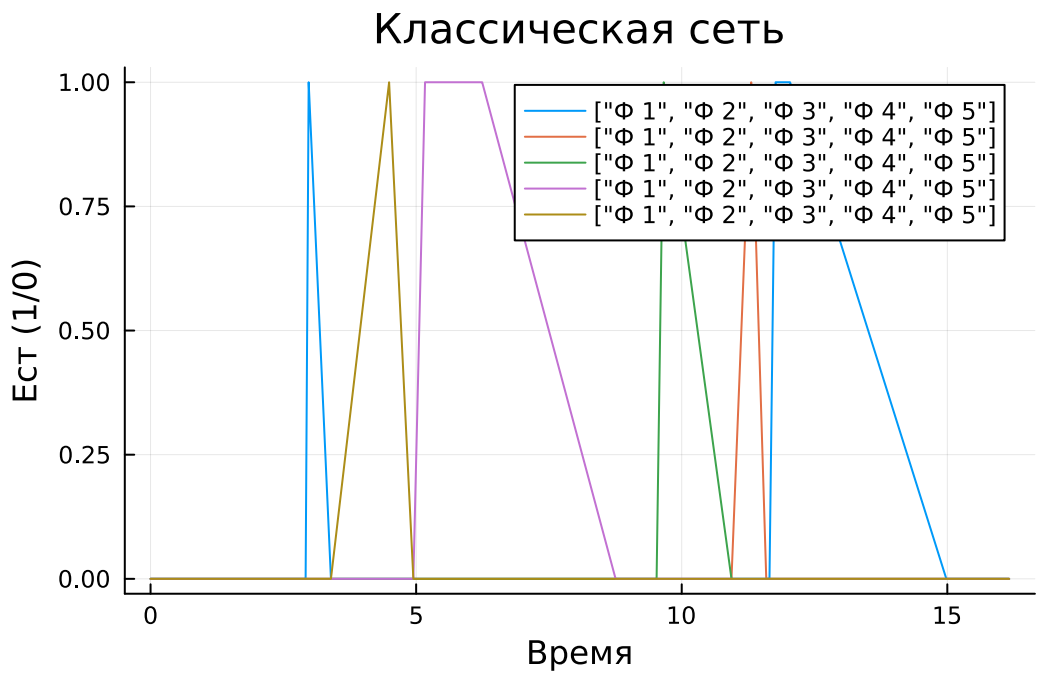
```
eat_cols = [Symbol("Eat_$i") for i = 1:N]
```

```
5-element Vector{Symbol}:
 :Eat_1
 :Eat_2
 :Eat_3
 :Eat_4
 :Eat_5
```

7.2. Построение графиков

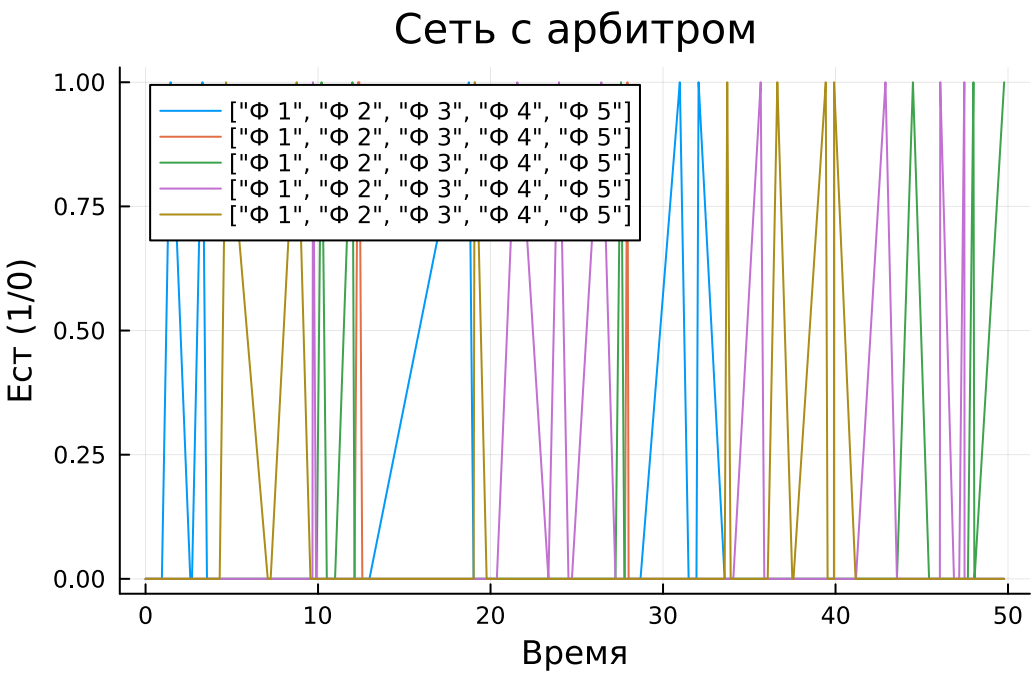
Первый график --- классическая сеть. По горизонтали --- время, по вертикали --- 1 или 0 (ест или нет). Каждый философ --- линия своего цвета.

```
p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["Ф $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)
```



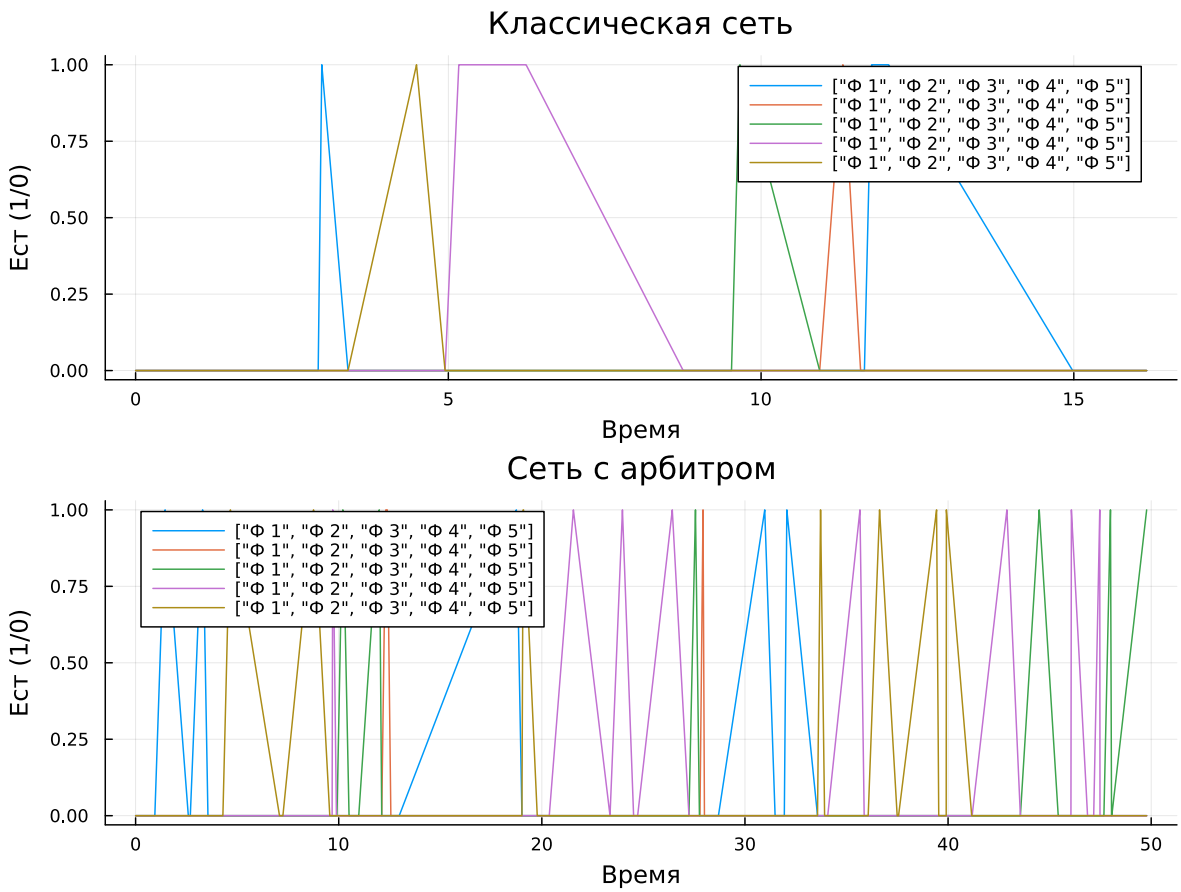
Второй график --- сеть с арбитром.

```
p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["Ф $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
)
```



Объединяем оба графика в один --- один над другим.

```
p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
```



7.3. Сохранение результата

Сохраняем сравнительный график в папку plots.

```
#savefig(plotsdir("final_report.png"))
```

Выводим сообщение об успешном сохранении.

```
println("Отчёт сохранён в plots/final_report.png")
```

```
Отчёт сохранён в plots/final_report.png
```